

Yes. This Reddit post is actually **quite useful for an MCA student**, especially because it comes from an LNCT student who claims experience with **15–20+ online assessments and 7 interviews**, with on-campus packages ranging from **₹4–16 LPA**. But treat it as **one student's experience, not official placement data**.

I've extracted the useful parts and converted them into something you can actually follow.

MCA Placement Strategy — What to Take From This Reddit Post

1. First understand the placement funnel

The most important insight is that **getting into the interview is itself a competition**.

A typical campus process looks like:

Eligibility → Shortlisting → Online Assessment → Technical Interview → HR/Final

The post says companies commonly use **60% or 70% academic eligibility**, while some use 80%. The author also says the first shortlisting at their college was based on CGPA, with some companies allowing thousands of students into the OA.

What this means for you

Don't think:

"I'll prepare seriously when placements start."

Instead:

Semester 1–2 → protect eligibility + build fundamentals

Semester 2–3 → DSA + projects + technical preparation

Semester 3–4 → placement-specific preparation

Your first objective is simply:

Never get eliminated because of something preventable.

2. DSA is non-negotiable

This is probably the strongest point in the post.

The author says you don't necessarily need the entire DSA universe, but recommends being strong **up to trees** and says DSA is essentially unavoidable for campus placements. They specifically mention LeetCode Top 150 as useful.

Your target

Don't obsess over:

- Advanced graph algorithms
- Competitive programming
- 500–1000 random problems

Instead build this progression:

Arrays

↓

Strings

↓

Hashing

↓

Two pointers / Sliding window

↓

Stack / Queue

↓

Linked List

↓

Recursion

↓

Binary Search

↓

Trees

↓

Basic graphs

Then solve interview-style problems.

Practical target

By placement season:

100–150 carefully selected problems > 500 problems solved mechanically.

The important thing is recognizing patterns.

"I like the footless bird, do you know why?"

"Because it has no legs, only wings, thus it has no choice but to fly. When it lands, that signifies its destruction."

3. Choose ONE primary language

This post has an interesting disagreement in the comments.

The original author strongly favors Java and says it gave him an advantage because interviewers asked heavily about Java/Spring Boot when they saw those technologies on his resume.

But another experienced commenter strongly disagrees with the idea that Python automatically means ML and says students should choose the language they actually want to use. They nevertheless warn that **Java was commonly expected in some campus OAs/interviews** at their college.

My interpretation

Don't blindly conclude:
Java = good, Python = bad.
That's too simplistic.
Instead:

For MCA placement, I'd recommend:

Primary: Java

Secondary: Python

Why?

Java gives you:

- OOP
- Collections
- Exception handling
- Multithreading basics
- Backend development
- Spring Boot
- Strong interview material

Python remains useful for:

- Automation
- AI/ML
- Data analysis
- Scripting
- Testing

So you don't need to abandon Python.

You can do:

DSA → Java

Projects/automation/AI → Python where appropriate

That's a particularly good combination for your background.

4. Your resume is basically your interview syllabus

This is one of the most important insights in the entire post.

The author says interviewers generally scan the resume and ask about what you've written. Therefore:

Don't put technologies on your resume that you cannot defend.

This changes how you should build your resume.

Bad approach:

Python, Java, C++, React, Node, AWS, Docker, Kubernetes, AI, ML, Cybersecurity, Spring Boot...

but you barely know any of them.

Better:

Java → strong

SQL → strong

Spring Boot → intermediate

Python → strong

Docker → practical

Project X → know every component

Then an interviewer sees:

Java

and asks:

"Explain HashMap."

You answer.

Then:

"Difference between ArrayList and LinkedList?"

You answer.

Then:

"What is inheritance?"

You answer.

Then:

"Tell me about your Spring Boot project."

You explain it.

Now the interview becomes much easier because **you control the territory**.

5. DBMS + SQL deserve high priority

The author specifically calls **DBMS the most important core subject**, including SQL.

For MCA, I'd make SQL a serious skill rather than something you study only for exams.

You should eventually be comfortable with:

- SELECT

- WHERE
- ORDER BY
- GROUP BY
- HAVING
- JOINS
- Subqueries
- Aggregate functions
- CASE
- CTEs
- Window functions
- Primary/foreign keys
- Normalization
- Transactions
- ACID
- Indexes

And, importantly, **actually write queries** rather than merely watching SQL tutorials.

6. Don't neglect the other core subjects

The post recommends preparing common interview questions for:

- DBMS
- OS
- Computer Networks
- Java
- React
- Spring Boot
- etc.

For MCA, I'd structure your core preparation as:

Priority	Subject
● Very High	DSA
● Very High	DBMS + SQL
● Very High	Java/OOP
● High	OS
● High	Computer Networks
● High	Software Engineering
● Medium	Web technologies
● Medium	Cloud/DevOps basics

You don't need university-level mastery of everything.

You need **interview-level command**.

7. Communication is a hidden filter

One commenter makes an important observation that the original post barely discusses:

Communication can make or break an interview.

This is particularly important because technical knowledge isn't useful if you cannot explain your solution.

You should practice explaining:

"Tell me about yourself."

"Explain your project."

"Why did you choose this technology?"

"What problem does your project solve?"

"What was the hardest bug you encountered?"

"Why should we hire you?"

You don't need polished corporate English.

You need:

clear + structured + understandable communication.

8. Aptitude/OA is the gatekeeper

This is another useful insight from the comments.

The commenter describes the **first OA as one of the hardest barriers** because a huge number of students compete there.

Therefore your preparation cannot be only:

DSA + projects.

You also need:

Quantitative aptitude

Logical reasoning

Verbal ability

Coding

Time management

This is particularly important because you have previously mentioned that aptitude/reasoning wasn't a major focus during your BCA. So for MCA, I'd deliberately fix that weakness early rather than waiting for placement season.

9. Internship: useful, but don't worship the certificate

This Reddit discussion is actually nuanced.

The original poster says they didn't do an internship and regrets it, but also says that for their campus placements, internships were **not necessarily required** and were more like an additional advantage.

So:

Don't chase:

"I need 5 internship certificates."

Instead chase:

"I need evidence that I can actually work."

One legitimate internship where you actually built/tested/deployed something is worth much more than five certificates from meaningless training programs.

10. Projects should solve real problems

One of the comments gives an excellent principle:

Build projects that solve actual problems instead of copying generic ML/MERN/Spring Boot projects from YouTube.

This connects directly with what **you were already thinking about**.

Your idea of a college resource/placement portal is actually better than another:

"Netflix clone"

or

"Weather app"

if you genuinely get students to use it.

Imagine your MCA project becoming:

College Student Knowledge Platform

Resources

- Syllabus
- Notes
- PYQs
- Lab manuals

Placement

- Company eligibility
- Packages
- Interview experiences
- Previous questions

Preparation

- DSA
- SQL
- Aptitude
- Mock interviews

Events

- Coding competitions
- Hackathons
- Technical workshops

Analytics

- Most searched companies
- Most requested topics
- Placement trends

Now you have:

Real users + real problem + deployment + database + authentication + analytics + project explanation

That's excellent interview material.

11. Your contest idea fits perfectly

Your earlier idea of organizing coding competitions is also supported by the Reddit discussion.

One commenter specifically advises juniors to:

participate in contests and focus on concepts rather than merely ratings.

Your role can be:

Technical organizer

- install/configure systems
- prepare environment
- coordinate faculty
- run competition
- maintain leaderboard
- publish results

You don't even need to know the questions beforehand if faculty controls the confidential question selection.

That gives you something useful for both:

College reputation + resume experience.

12. Use seniors as an information source

This is not emphasized enough in the post, but the comment section reveals something valuable: students repeatedly ask seniors about:

- language selection
- internships
- shortlisting
- placements
- eligibility
- preparation

You can formalize this.

Your college website could eventually contain:

Company → Interview Database

Company	Eligibility	OA	Technical	HR	Package	Experience
TCS	—	Aptitude + Coding	CS fundamentals	Yes	—	2025
Company B	—	Coding	Java + DBMS	Yes	—	2025

That becomes extremely valuable when your placement season arrives.

Your MCA plan from this Reddit post

If I convert everything into **one simple roadmap**, I'd make it:

Semester 1

Primary

- Maintain good CGPA
- Java fundamentals
- SQL
- OOP
- Basic Git/GitHub

Secondary

- Explore development/AI/testing
- Participate in contests

College contribution

- Start collecting placement/company information
- Start building the student resource platform

Semester 2

Primary

- DSA seriously
- Java Collections
- DBMS
- SQL
- OS basics

Projects

- Build and deploy your college resource/placement platform

Activities

- Participate in coding contests
- Organize one small technical competition

Semester 3

This becomes **placement mode**.

DSA

- Trees
- Graph basics
- Interview patterns

- ~100–150 quality problems

Core

- DBMS
- OS
- CN
- Java/OOP

Projects

- Know your projects extremely deeply.

Interview

- Mock interviews
- Introduction
- Project explanation
- Technical questions
- HR questions

Semester 4

Don't learn 20 new technologies.

Instead:

Apply → OA → Interview → analyze failure → improve → repeat.

And simultaneously apply outside campus.

The biggest lesson I take from this post

The post isn't really saying:

"Learn Java."

or

"Solve LeetCode."

The deeper lesson is:

Don't wait for placement season to become employable.

By the time companies arrive, you should already have:

CGPA

- **DSA**
- **Java**
- **SQL/DBMS**
- **OS/CN**
- **2 strong projects**
- **communication**
- **aptitude**
- **some contest/internship/real-world experience**

Then placement season becomes an **execution phase**, not a learning-from-zero phase.

And your idea of using the college itself as a platform — collecting information, building useful student services, organizing contests, connecting with seniors and faculty — is actually a very good complement to this strategy.

It gives you something that a normal "study DSA + make projects" student doesn't have: **institution-specific information and relationships.**